Bases de la programmation

Épreuve de TP — Sujet A : gestion des notes d'une promotion

Durée : 1 h 30 • Examen blanc (entraînement)

Consignes. Épreuve sur machine. Tout votre travail est à rendre dans **un seul fichier** `NOM_Prenom_tpA.py`, déposé à la fin de l'épreuve. Un programme qui ne s'exécute pas (erreur de syntaxe) sera fortement pénalisé : testez chaque fonction au fur et à mesure et commentez ce qui ne fonctionne pas plutôt que de le laisser. Les appels de test sont regroupés dans une fonction `main()`, appelée en fin de fichier par `if __name__ == "__main__":`.

Pour les listes, seules les opérations `len`, `append` et l'accès par indice `[]` sont autorisées (ni `in`, ni `sum`, `max`, `min`, `sort`, `sorted`, tranches…). Accès autorisé à la documentation Python officielle uniquement ; Internet, IA et messageries interdits.

On souhaite écrire un petit logiciel qui gère les notes d'une promotion. Les notes sont rangées dans une matrice `M` : la ligne `i` correspond à l'étudiant numéro `i` et la colonne `j` à la matière numéro `j`. Comme en TD, toute fonction qui reçoit la matrice reçoit aussi `n` (nombre d'étudiants) et `m` (nombre de matières).

Exercice 1 — Saisie et mention *4 points*

1. Écrire une fonction `saisir_note()` qui demande une note réelle à l'utilisateur, la redemande tant qu'elle n'est pas comprise entre 0 et 20 (inclus), puis la renvoie.
2. Écrire une fonction `moyenne(L)` qui renvoie la moyenne des valeurs de la liste `L` (on suppose `L` non vide).
3. Écrire une fonction `mention(moy)` qui renvoie la chaîne correspondant à la moyenne : « Ajourné » (moins de 10), « Passable » (de 10 inclus à 12 exclu), « Assez bien » (de 12 à 14), « Bien » (de 14 à 16) ou « Très bien » (16 et plus).

Exercice 2 — La matrice des notes *5 points*

1. Écrire une fonction `generer_notes(n, m)` qui crée et renvoie une matrice de `n` lignes et `m` colonnes remplie d'entiers tirés au hasard entre 0 et 20 (`from random import randint`).
2. Écrire une fonction `afficher(M, n, m)` qui affiche la matrice avec un numéro d'étudiant en début de ligne :

```
Étudiant 0 : 12 8 15
Étudiant 1 : 5 11 9
Étudiant 2 : 18 14 16
```

3. Écrire une fonction `moyenne_etudiant(M, n, m, i)` qui renvoie la moyenne de l'étudiant `i`, et une fonction `moyenne_matiere(M, n, m, j)` qui renvoie la moyenne de la promotion dans la matière `j`. Vous pouvez réutiliser `moyenne`.
4. Écrire une fonction `bilan(M, n, m)` qui affiche, pour chaque étudiant, sa moyenne (arrondie à 2 décimales avec `round(x, 2)`) et sa mention.

Exercice 3 — Statistiques *4 points*

1. Écrire une fonction `meilleur_etudiant(M, n, m)` qui renvoie deux valeurs : le numéro de l'étudiant ayant la meilleure moyenne (le premier en cas d'égalité) et cette moyenne.
2. Écrire une fonction `nb_admis(M, n, m)` qui renvoie le nombre d'étudiants dont la moyenne est supérieure ou égale à 10.
3. Écrire une fonction `matiere_la_plus_difficile(M, n, m)` qui renvoie le numéro de la matière ayant la plus faible moyenne.

Exercice 4 — Classement *4 points*

On veut afficher le classement des étudiants, de la meilleure moyenne à la moins bonne.

1. Écrire une fonction `liste_moyennes(M, n, m)` qui renvoie la liste des moyennes des `n` étudiants.
2. Écrire une fonction `classement(M, n, m)` qui renvoie la liste des numéros d'étudiants triés par moyenne *décroissante*. On utilisera un *tri à bulles* sur deux listes en parallèle : la liste des moyennes et la liste des numéros `[0, 1, ..., n-1]`. Chaque fois que deux moyennes voisines sont échangées, les deux numéros correspondants sont échangés aussi.
3. Écrire une fonction `afficher_classement(M, n, m)` qui affiche le classement :

```
1er : étudiant 2 (16.0)
2e : étudiant 0 (11.67)
3e : étudiant 1 (8.33)
```

Exercice 5 — Programme principal *3 points*

Écrire la fonction `main()` qui demande le nombre d'étudiants et le nombre de matières (entiers strictement positifs, saisie contrôlée), génère la matrice des notes, puis affiche en boucle le menu suivant jusqu'à ce que l'utilisateur choisisse 0 :

```
1 - Afficher les notes
2 - Bilan par étudiant
3 - Statistiques (meilleur étudiant, admis, matière la plus difficile)
4 - Classement
5 - Modifier une note
0 - Quitter
Votre choix :
```

Un choix invalide affiche « Choix inconnu ». Le choix 5 demande un numéro d'étudiant et un numéro de matière valides, puis la nouvelle note (avec `saisir_note`).

Bonus (+1). Ajouter un choix 6 qui affiche, pour chaque matière, la moyenne, la note minimale et la note maximale de la promotion.

Bases de la programmation

Épreuve de TP — Sujet B : le jeu du morpion

Durée : 1 h 30 • Examen blanc (entraînement)

Consignes. Épreuve sur machine. Tout votre travail est à rendre dans **un seul fichier** `NOM_Prenom_tpB.py`, déposé à la fin de l'épreuve. Un programme qui ne s'exécute pas (erreur de syntaxe) sera fortement pénalisé : testez chaque fonction au fur et à mesure et commentez ce qui ne fonctionne pas plutôt que de le laisser. Le programme est lancé par une fonction `main()`, appelée en fin de fichier par `if __name__ == "__main__":`.

Pour les listes, seules les opérations `len`, `append` et l'accès par indice `[]` sont autorisées (ni `in`, ni `count`, `index`, tranches…). Accès autorisé à la documentation Python officielle uniquement ; Internet, IA et messageries interdits.

Le morpion se joue à deux sur une grille carrée. Chacun à son tour, un joueur place son symbole (`X` pour le joueur 1, `O` pour le joueur 2) dans une case vide. Le premier qui aligne `taille` symboles sur une ligne, une colonne ou une diagonale gagne ; si la grille est pleine sans gagnant, la partie est nulle.

La grille est une matrice carrée `G` de taille `t` × `t` (`t = 3` pour le morpion classique). Chaque case contient `0` (vide), `1` (joueur 1) ou `2` (joueur 2). Toutes les fonctions doivent fonctionner pour n'importe quelle taille `t >= 3`.

Exercice 1 — La grille *4 points*

1. Écrire une fonction `grille_vide(t)` qui crée et renvoie, à l'aide de `append`, une grille `t` × `t` remplie de 0.
2. Écrire une fonction `symbole(v)` qui renvoie `"."` si `v` vaut 0, `"X"` s'il vaut 1 et `"O"` s'il vaut 2.
3. Écrire une fonction `afficher(G, t)` qui affiche la grille avec les numéros de lignes et de colonnes. Exemple pour `t = 3` :

```
  0 1 2
0 X . O
1 . X .
2 . . O
```

Exercice 2 — Jouer un coup *4 points*

1. Écrire une fonction `coup_valide(G, t, i, j)` qui renvoie `True` si la case `(i, j)` existe dans la grille *et* est vide, `False` sinon.
2. Écrire une fonction `saisir_coup(G, t, joueur)` qui affiche le symbole du joueur, demande un numéro de ligne puis un numéro de colonne, et recommence tant que le coup n'est pas valide (en affichant « Coup invalide »). Elle renvoie les deux valeurs `i` et `j`.
3. Écrire une fonction `grille_pleine(G, t)` qui renvoie `True` si aucune case n'est vide.

Exercice 3 — Détection du gagnant *5 points*

1. Écrire une fonction `ligne_gagnante(G, t, i, joueur)` qui renvoie `True` si toutes les cases de la ligne `i` appartiennent à `joueur`.
2. Écrire de même `colonne_gagnante(G, t, j, joueur)`.
3. Écrire une fonction `diagonale_gagnante(G, t, joueur)` qui teste les *deux* diagonales (la diagonale principale contient les cases `(k, k)`, l'autre les cases `(k, t-1-k)`).
4. Écrire une fonction `a_gagne(G, t, joueur)` qui renvoie `True` si `joueur` a gagné, en utilisant les trois fonctions précédentes.

Exercice 4 — Une partie *4 points*

1. Écrire une fonction `partie(t)` qui joue une partie complète entre deux joueurs humains : elle crée la grille, puis, tant que la partie n'est pas terminée, affiche la grille, fait jouer le joueur courant et change de joueur. À la fin, elle affiche la grille et le résultat (« Le joueur X a gagné » ou « Match nul »), et renvoie le numéro du gagnant (1 ou 2), ou 0 en cas de match nul.
2. Écrire la fonction `main()` qui demande la taille de la grille (entier entre 3 et 6, saisie contrôlée), enchaîne les parties tant que l'utilisateur répond exactement `"o"` à la question « Une autre partie (o/n) ? », et tient le score. Avant de s'arrêter, elle affiche le nombre de victoires de chaque joueur et le nombre de matchs nuls.

Exercice 5 — Contre l'ordinateur *3 points*

On veut maintenant que le joueur 2 soit l'ordinateur.

1. Écrire une fonction `coup_aleatoire(G, t)` qui renvoie une case vide tirée au hasard (on tire `i` et `j` avec `randint` jusqu'à obtenir une case vide ; on suppose la grille non pleine).
2. Écrire une fonction `coup_ordinateur(G, t)` un peu plus maligne : si l'ordinateur peut gagner en un coup, il joue ce coup ; sinon, si le joueur 1 peut gagner au coup suivant, il bloque cette case ; sinon, il joue au hasard.
   Indication : pour tester une case vide, on y place provisoirement le joueur, on appelle `a_gagne`, puis on remet la case à 0.
3. Modifier `partie` (ou en écrire une copie `partie_ordi`) pour que le joueur 2 utilise `coup_ordinateur`, et proposer ce mode de jeu dans `main`.